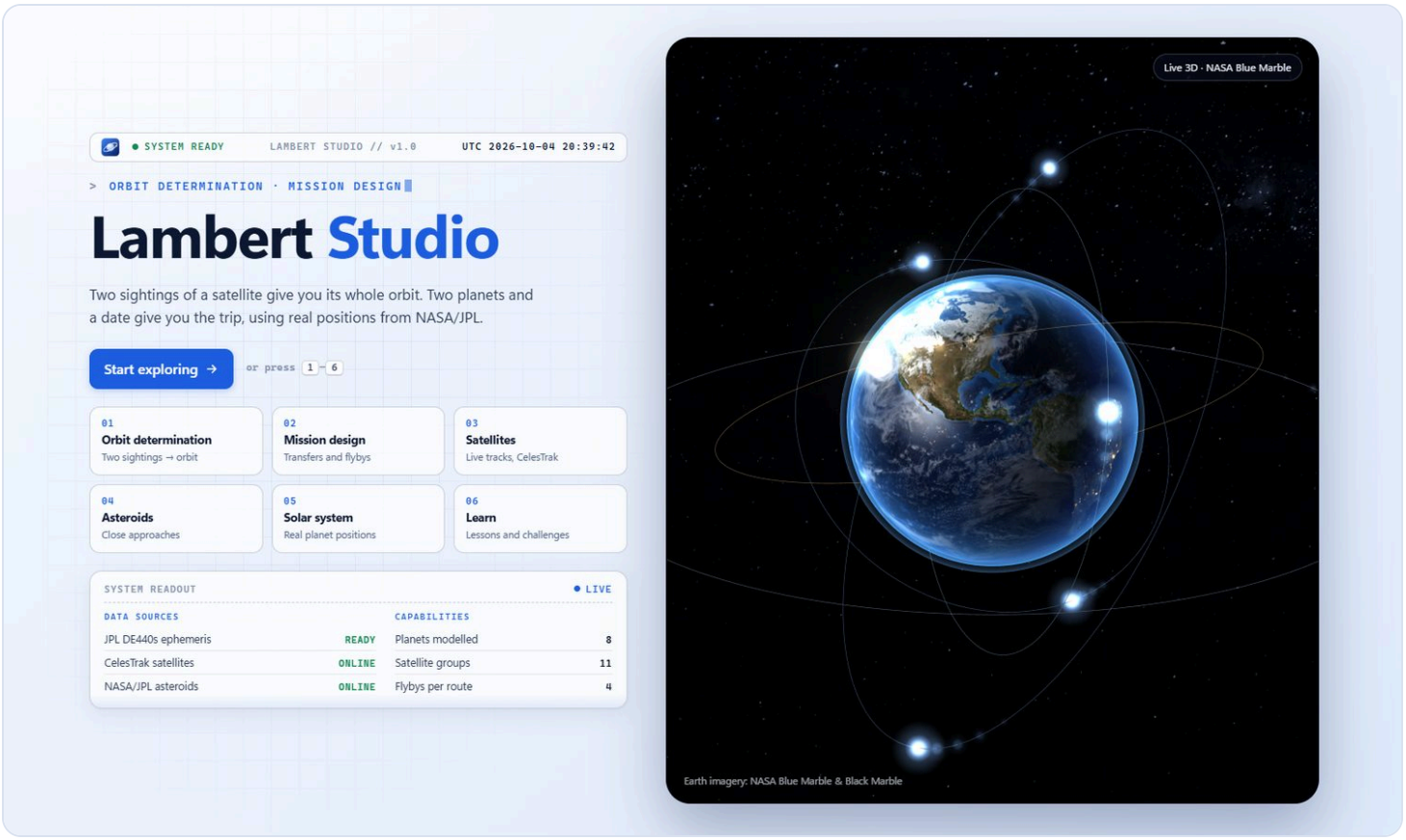
PROJECT REPORT · 4 OCTOBER 2026

# Lambert Studio

An application for orbit determination and interplanetary mission design, built on Lambert's problem. This report records how it was built, what was found along the way, how well it performs, and how to present it.

Version 1.0  Windows desktop app and Python source

Physics validated against textbook and NASA/JPL references




The start page. The Earth turns in real 3D; the readout on the left shows live data-source status and the UTC clock.

## Summary

**Lambert Studio** turns two measurements into an answer. Given two sightings of a satellite from a ground station, it finds the satellite's complete orbit. Given two planets and a launch date, it plans the transfer, including routes that use gravity assists, and it draws everything in 3D with real planetary positions from NASA/JPL.

It grew from a university assignment on **Lambert's problem** (Curtis, *Orbital Mechanics for Engineering Students*). The assignment deliverable, a single Python file, is kept and still passes its own self-test. The Studio is the extended version: a desktop application with a web-based

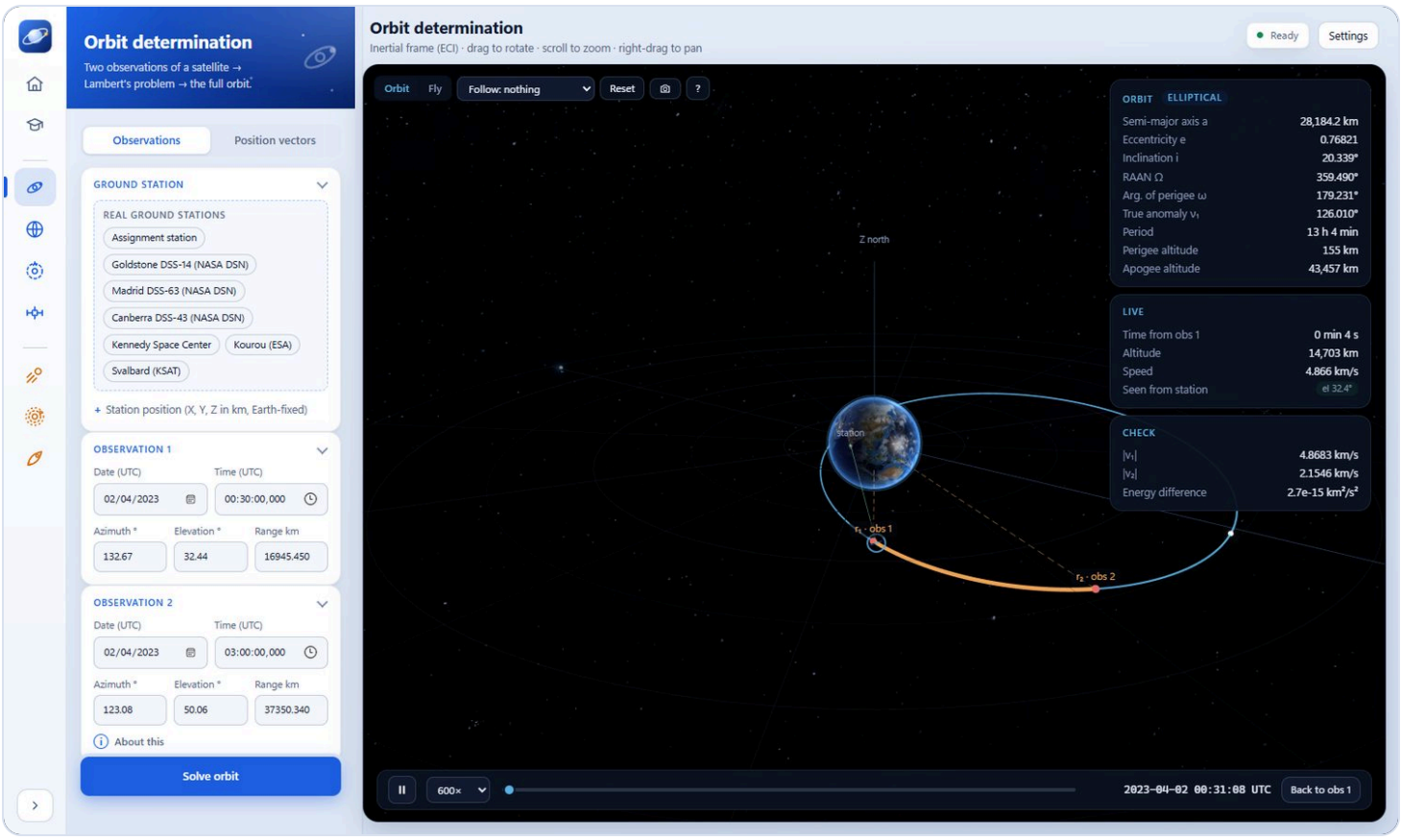
interface, a physics engine in Python, live satellite and asteroid data, an interactive Learn section, and a standalone Windows build that runs without Python.

| | | |
|---|---|---|
| VALIDATION<br>**117 checks**<br>all passing; 83 run inside the single-file hand-in | AUTOMATED TESTS<br>**39**<br>17 physics groups, 22 server API tests | VIEWS<br>**8 + start page**<br>orbit, ground track, maneuvers, satellites, asteroids, solar system, missions, learn |
| CODE<br>**~7,100 lines Python**<br>~6,500 lines JavaScript, ~900 CSS | SERVER API<br>**23 routes**<br>local only, 127.0.0.1 | STANDALONE APP<br>**186 MB**<br>107 MB zipped, no Python needed |

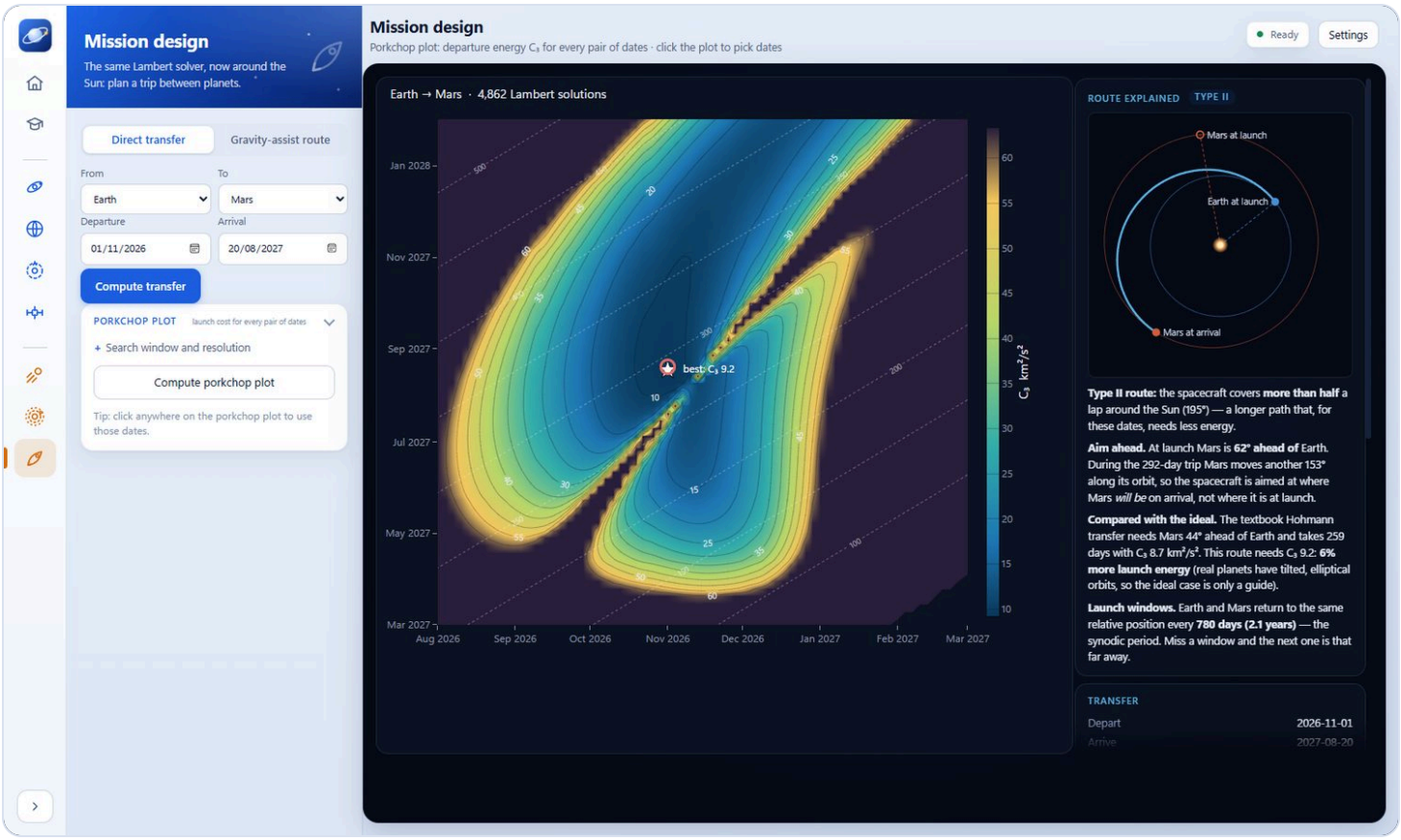
## The application

Every view shares one layout: a labelled control panel on the left, a 3D or chart area on the right, and a time bar for anything that moves. The sidebar groups the views by topic, blue for Earth orbit and amber for deep space.

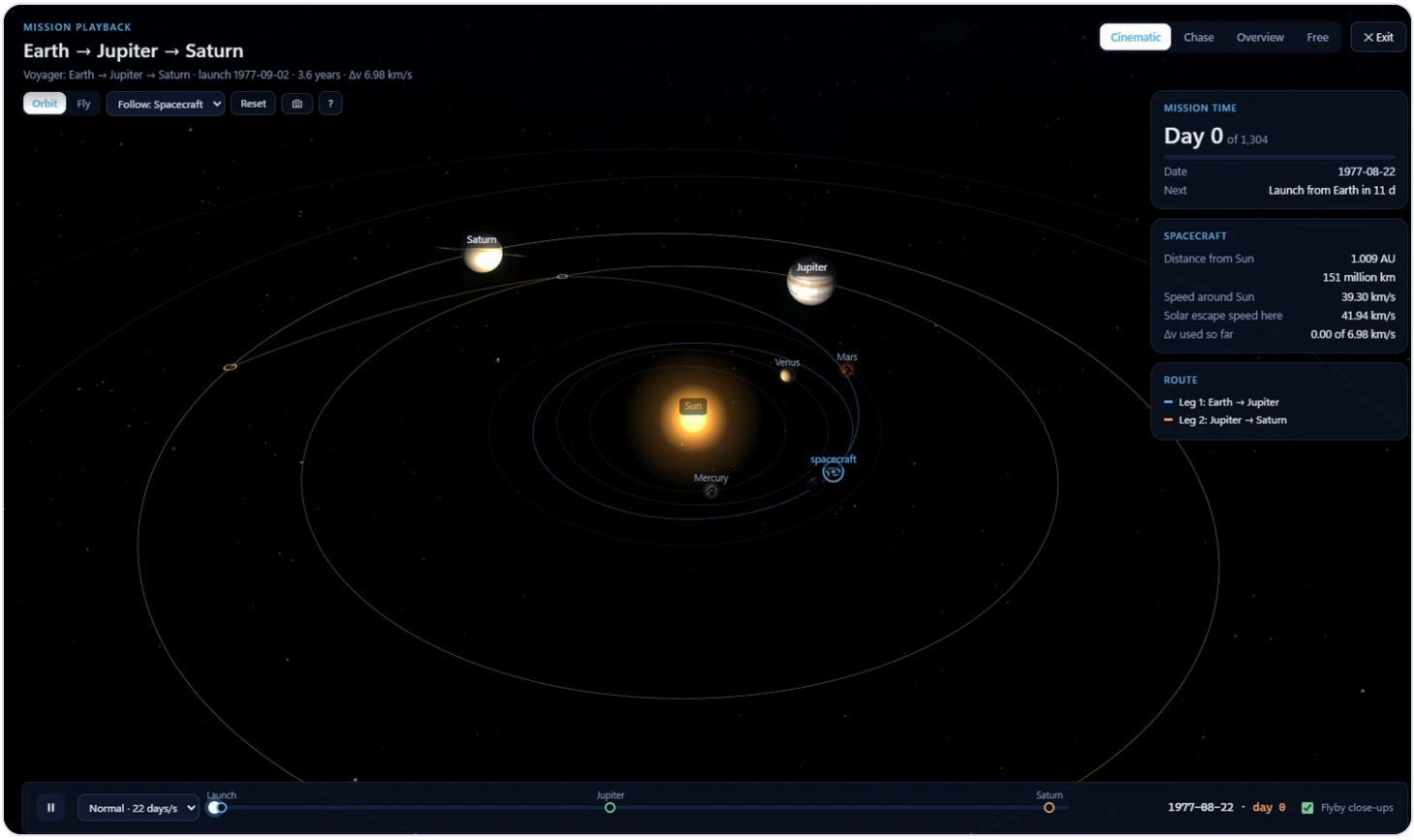
| VIEW | WHAT YOU DO | WHAT YOU SEE |
|---|---|---|
| Orbit determination | Enter two sightings (azimuth, elevation, range) or two position vectors | The full orbit in 3D, its elements, live altitude and speed, an uncertainty study |
| Ground track | Use the solved orbit | The track over a world map and the station's sky view on one timeline |
| Maneuvers | Pick Hohmann, bi-elliptic or plane change | Delta-v per burn, propellant, animated transfer, comparison of options |
| Satellites | Switch constellations on and off, search by name | About 900 real satellites at their current positions (CelesTrak) |
| Asteroids | Choose an upcoming close approach | The flyby path past Earth and the Moon (NASA/JPL CNEOS and Horizons) |
| Solar system | Focus a planet, toggle true-scale distances | The planets at their real positions (JPL DE440s) and 2,549 hazardous asteroids |
| Mission design | Pick two planets and dates, or a gravity-assist route | A porkchop plot, a plain-language route explanation, and a full-screen mission playback |
| Learn | Read eight short lessons, answer quizzes, try challenges | Interactive diagrams and automatically checked challenges |



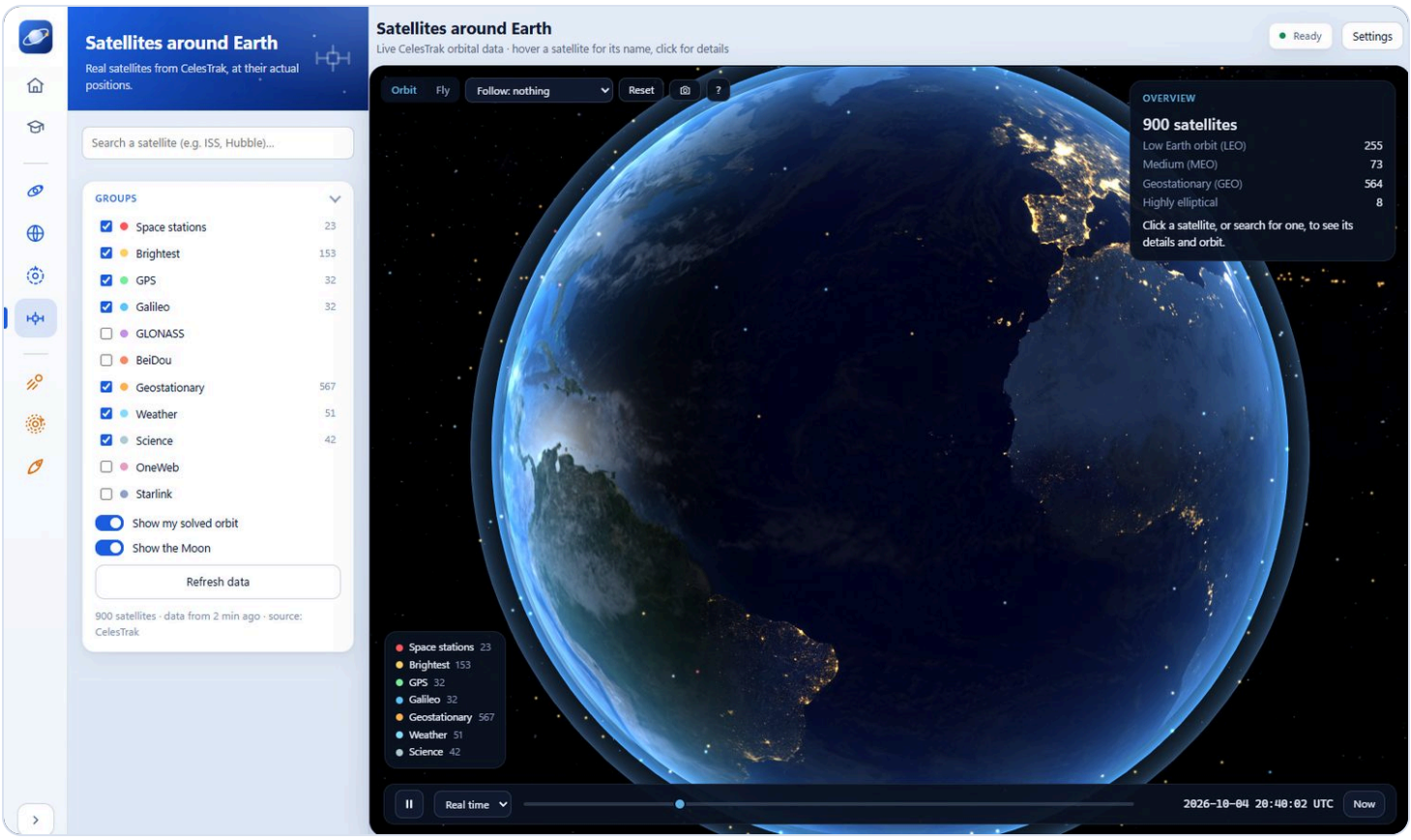

Orbit determination: the orbit solved from two sightings, with its elements and live values.



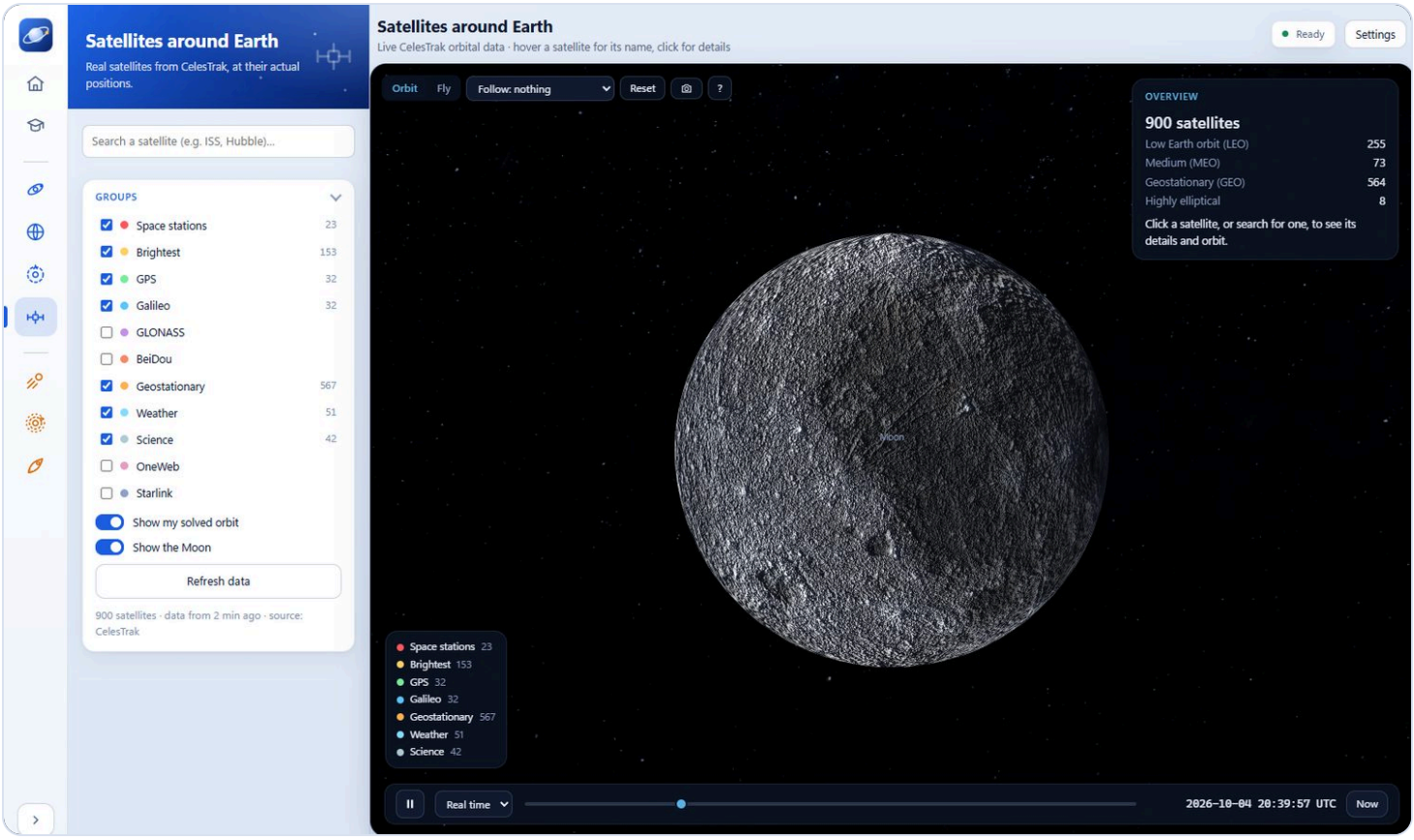

Mission design: 4,862 independent Lambert solutions make the porkchop plot; the dark eye is the cheapest launch window.



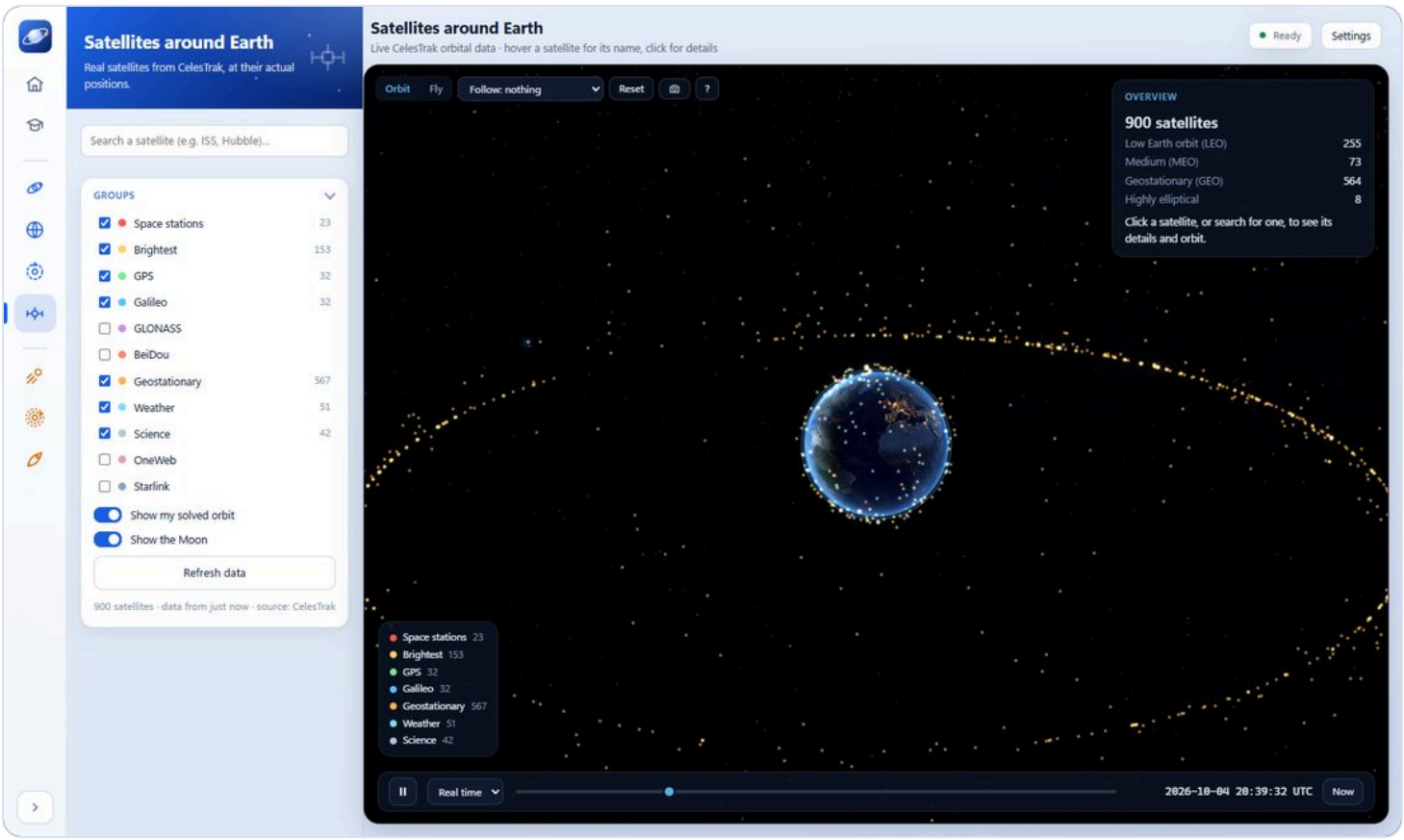

Mission playback of the Voyager route (Earth, Jupiter, Saturn), with mission time, spacecraft speed against solar escape speed, and the Delta-v used so far.



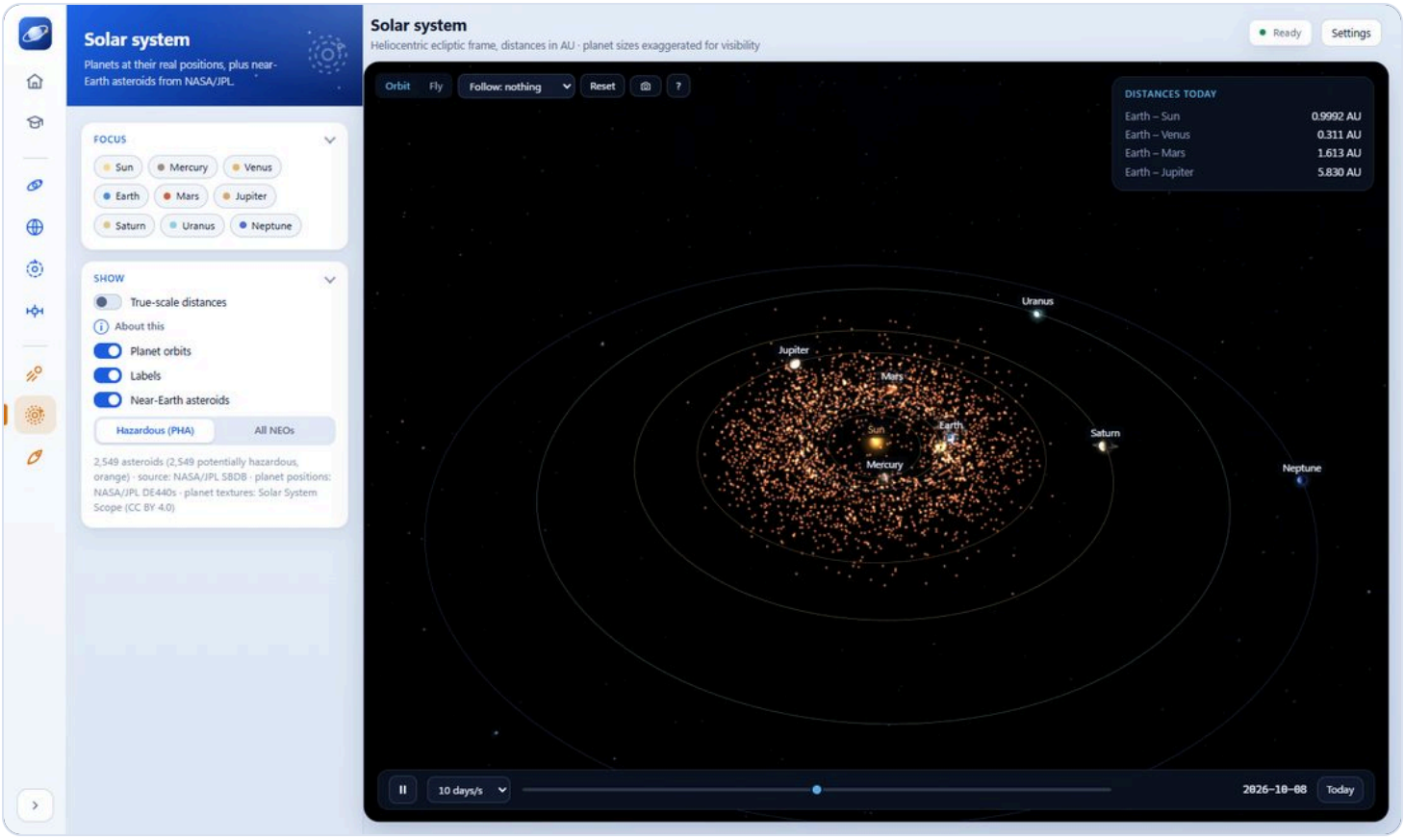

Earth from low altitude: NASA Black Marble city lights under cloud, with the atmosphere's haze at the limb.



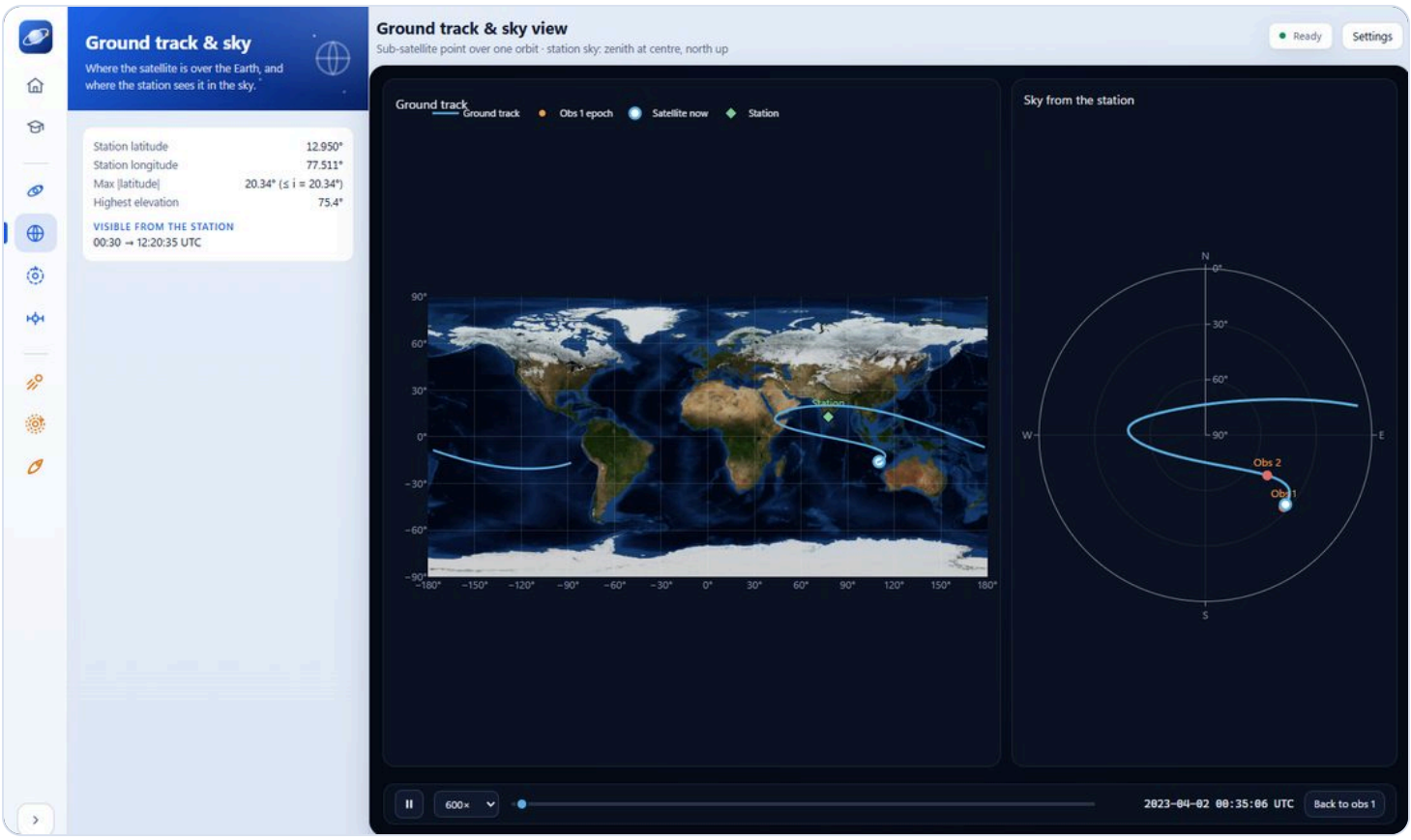

The Moon from NASA's Lunar Reconnaissance Orbiter data, with relief shading.



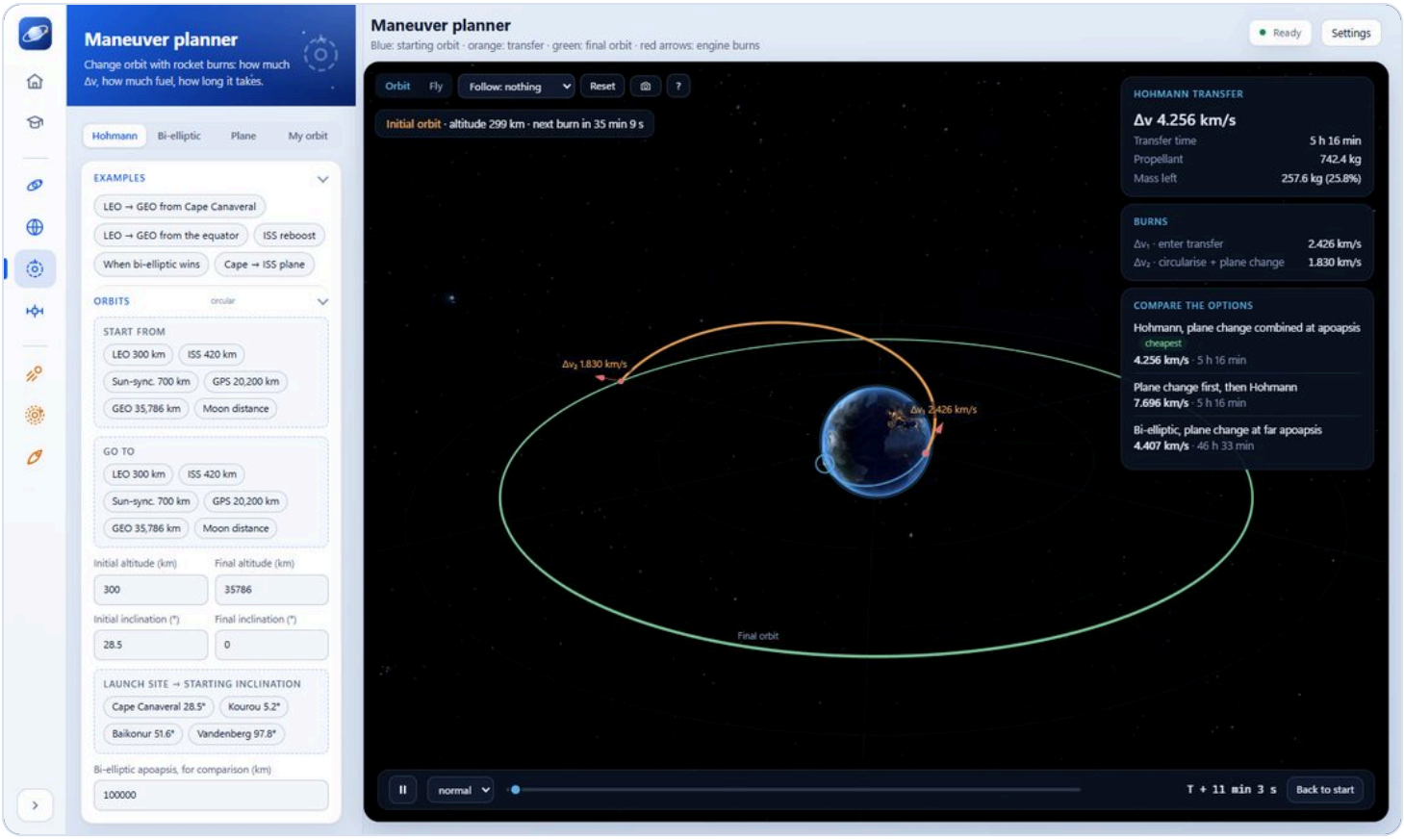

Satellites



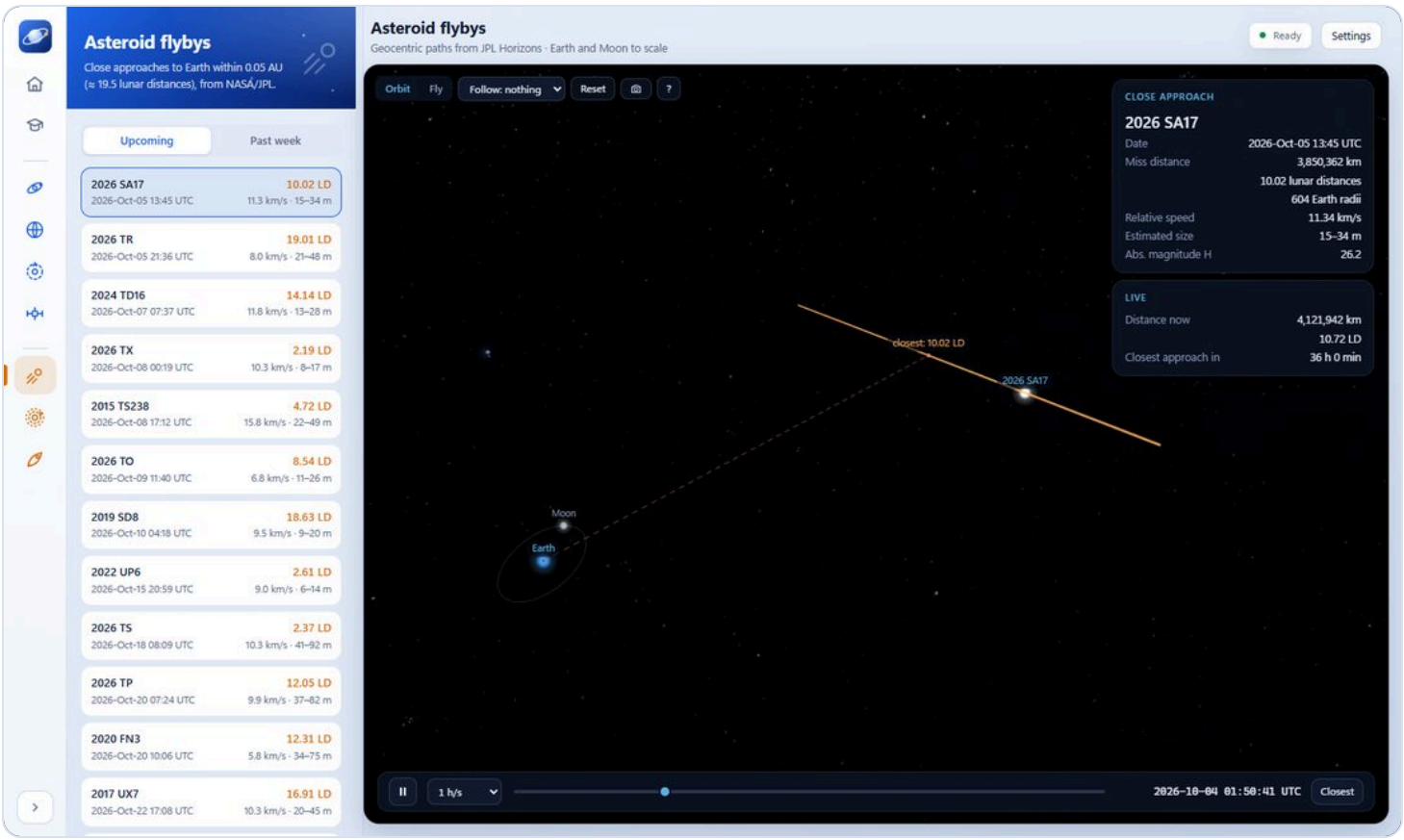

Solar system



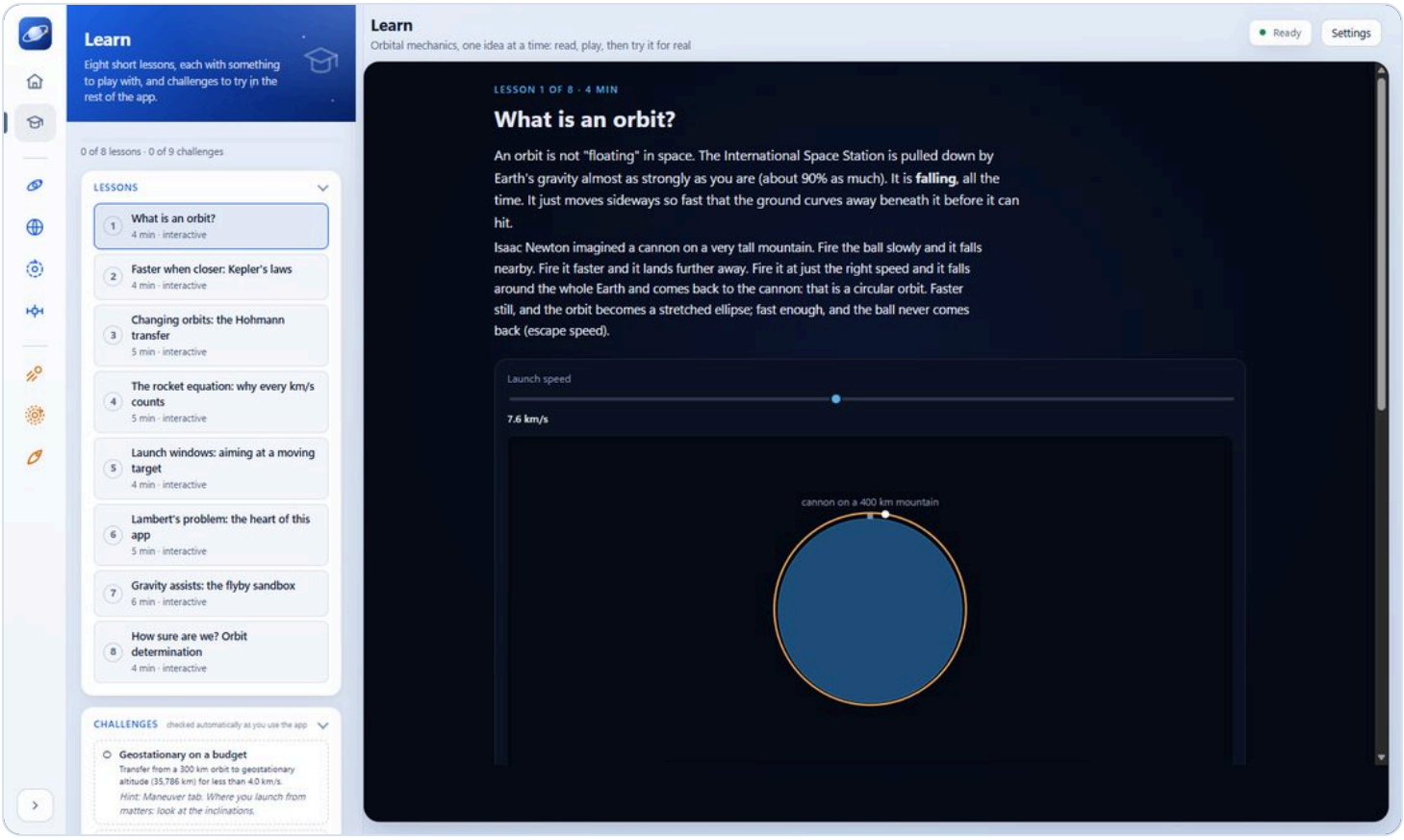

Ground track and sky




Maneuver planner




Asteroid flybys




Learn section

## How it works

The physics lives in Python and the picture lives in the browser. A small local server connects them. Nothing leaves the computer except requests for public data.

**INTERFACE**

HTML, CSS and JavaScript. Three.js draws the 3D scenes on one shared renderer; Plotly draws the charts.

↕

**LOCAL SERVER**

23 JSON routes on 127.0.0.1. Validates every input and returns plain errors a person can act on.

↕

**PHYSICS ENGINE**

Python with NumPy and SciPy: Lambert solver, orbit elements, patched-conic flybys, an n-body check.

`JPL DE440s ephemeris` `CelesTrak satellite elements` `JPL Small-Body Database`

`JPL CNEOS close approaches` `JPL Horizons paths`

The app opens in an Edge window (app mode, no browser chrome). On the standalone build the same code runs from a packaged folder with its own Python, so the audience needs nothing installed.

### The methods

| PART | METHOD |
|---|---|
| Lambert's problem | Universal-variable formulation (Curtis, Algorithm 5.2) with Stumpff functions and a bracketed root finder; multi-revolution solutions for long transfers |
| Orbit from sightings | Azimuth, elevation and range become a position in the station frame, then in the Earth-fixed and inertial frames; two positions and the time between them feed the solver |
| Planet positions | NASA/JPL DE440s through Skyfield, with a built-in approximation if the file is missing |
| Interplanetary transfers | The same Lambert solver around the Sun; launch energy C3, arrival speed, capture burn; porkchop plots over a grid of dates |
| Gravity assists | Patched conics, up to four flybys, loops around the Sun allowed, optimisation of dates, plus a deep-space burn variant (Juno-style) |
| Course check | The designed route is flown again under full solar-system gravity with trajectory-correction burns |
| Satellites | Kepler motion with the J2 secular drift, compared against SGP4 in the validation suite |
| Uncertainty | Monte Carlo: the orbit is re-solved thousands of times with random measurement errors |

## Validation

The validation suite ( `src/validation.py` ) holds 117 checks. They compare results against the textbook example, against known mission figures, and against physical invariants. All pass.
These are the results a reviewer is most likely to ask about.

| CHECK | RESULT | REFERENCE OR CRITERION |
|---|---|---|
| Assignment orbit, semi-major axis | 28,184.19 km | Matches the assignment data |
| Eccentricity, inclination | 0.768209 · 20.339° | Matches the assignment data |
| RAAN, argument of perigee, true anomaly | 359.490° · 179.231° · 126.010° | Matches the assignment data |
| Period, perigee and apogee altitude | 13 h 04 min · 155 km · 43,457 km | Matches the assignment data |
| Mars transfer 2026, total delta-v | 5.6704 km/s | Design value 5.6788 km/s, tolerance 0.05 km/s |
| Voyager, total course corrections | 4.2 m/s | Criterion: below 50 m/s |
| Voyager, Saturn closest approach after corrections | 12,054 km | Equals the planned value within 50 km |
| Mars trip flown uncorrected in full gravity | misses by >10,000 km | After corrections it arrives within 10 km of the planned altitude |
| Juno deep-space burn route | Earth flyby within a month | Real flyby: 9 October 2013; launch C3 close to the real 31.1 km²/s² |
| No measurement error, uncertainty study | spread = 0 | Doubling the angle error doubles the spread |

The same run is available to anyone with the source: `python src/validation.py`. The single-file hand-in carries its own copy ( `python course/lambert_problem_solver.py --selftest` , 83 of the checks).

## Build process

Work on the Studio ran from 29 September to 4 October, from a working assignment to a packaged application. The history below comes from the project's git log and the working notes.

**29 SEP**

**Safe start**

Backed up the original folder, restored missing modules, and kept the assignment's single-file build as the hand-in path. Built the new interface (Python server, Three.js and Plotly front end) with the solar system, satellites and asteroids.

**30 SEP**

**Mission design grows**

Multi-revolution Lambert solutions, repeat flybys (Galileo, Cassini presets), mission playback, route suggestions with launch windows, a flyby close-up, the Learn section, deep-space burns, launch-vehicle curves and an uncertainty study. First case-study report and LinkedIn carousel.

**1 OCT**

**Fidelity and reports**

Higher-fidelity forces and a full solar-system gravity check. Reports in English and Spanish. A full input sweep of every view found and fixed a set of input errors.

**3 OCT**

**Focus and redesign**

Removed three views that were more than the project needed. Redesigned the interface (bright theme, start page, icon sidebar, section cards), reorganised the files into `src`, `web`, `course`, `tools`, added an app icon, built the standalone Windows app, and raised 3D quality with NASA textures.

**4 OCT**

**Performance and this report**

Found and fixed the cause of lag on the laptop's integrated graphics, and wrote this report.

## Decisions that shaped it

| DECISION | REASON |
| --- | --- |
| Keep the single-file hand-in alongside the Studio | The assignment requires one Python file; the generated file is rebuilt from the same modules and tested on its own |
| Python for physics, a web page for the interface | NumPy and SciPy for the numerics; Three.js and Plotly for graphics that a desktop toolkit cannot match |
| Edge app-mode window instead of an embedded Qt browser | The Qt web engine crashed on this machine; Edge is kept current by Windows and uses the dedicated graphics card |
| App data in the user's home folder | The Microsoft Store build of Python silently redirects writes under AppData into a hidden package folder |
| One shared 3D renderer for every view | Browsers allow about 16 graphics contexts per page and drop the oldest; separate renderers per view caused reloads |
| Remove Orbit prediction, Arrival and Ion engines | Scope: the project reads better with fewer, stronger views |
| Dark 3D windows inside a bright interface | Planets and orbits need a dark background; the surrounding interface stays light and calm |

## Findings and fixes

This table lists the problems found during the work, with their cause and outcome. Entries marked `OPEN` are still unresolved or only partly verified.

| PROBLEM | CAUSE | OUTCOME | STATUS |
|---|---|---|---|
| Qt window crashed with a native error | A fault inside Qt WebEngine on this machine | Edge app mode is the default; Qt stays as an option | FIXED |
| PySide6 failed to install in the default Python | Windows 260-character path limit with the Store build of Python | Project uses its own environment in the user folder | FIXED |
| Saved data seemed to vanish | Store Python redirects AppData writes | Data lives in `.lambert_studio` in the home folder | FIXED |
| A physics library aborted the app when started without a console | A Fortran runtime exits when it has no output stream | Launcher gives it output files; the feature was later removed with its view | FIXED |
| Orbit view reloaded the whole app after several tabs | More than about 11 graphics contexts; the browser dropped one | One shared renderer for all views | FIXED |
| Window opened partly off screen | Edge remembered a window position from an earlier run that was larger than the usable screen | Launcher fits the window to the screen and clears the saved position | FIXED |
| "Translate this page?" popup over the start page | The system language is Spanish, the page is English | Translation switched off for the app profile | FIXED |
| Start page could be skipped by a stray Enter key | A keyboard shortcut I added reacted to Enter | Shortcut removed; keys 1 to 6 remain | FIXED |
| The app lagged after the quality upgrade | Edge ran on the integrated Intel chip; 2x resolution plus 8x smoothing; every view kept its own large glow buffers; blur effects over moving 3D | Launcher requests the RTX 4070; one shared glow pipeline; quality adapts to the frame rate; blur removed; a Graphics quality setting | FIXED |
| Speed menu unreadable (white on white) in the time bar and playback | Controls outside the dark 3D area picked up the light theme | Dark palette scoped to the time bar and playback | FIXED |

| PROBLEM | CAUSE | OUTCOME | STATUS |
|---|---|---|---|
| Report study script crashed after the view removals | It depended on the removed force model | Script now stops with a message saying how to restore the files; saved results unaffected | FIXED |
| An intermittent "Error" on the Galileo route in one headless run | Not found; not reproduced in more than ten retries | Request logging and a retry were added | OPEN |
| Quick close and relaunch of the exe sometimes closed the window within seconds | Seen only when the app was killed and restarted back to back; a clean launch stayed open for 40 seconds | Wait a few seconds between closing and reopening | OPEN |

**Testing lessons.** A screenshot tool that is not display-scaling aware captured only the top-left 80% of the screen and made a correct window look oversized. Closing every Edge process by name also closed the user's own browser. Both are avoided now: captures declare DPI awareness, and only the app's own profile is touched.

## Performance

The laptop has an Intel UHD chip and an NVIDIA RTX 4070. Windows gave Edge the Intel chip by default, so the app now asks for the RTX at launch. Frame rate was measured in a hidden Edge window of 1,500 by 900 pixels, five readings per view, with the app's own frame counter. The counter tops out at 144 frames per second.

| SETUP | GRAPHICS CHIP | DRAWING RESOLUTION | LANDING | ORBIT | SATELLITES | S |
|---|---|---|---|---|---|---|
| Current, Auto | Intel UHD | 1.0× | 144 | 144 | 144 | |
| Current, Auto | RTX 4070 | 2.0× | 144 | 142–144 | 144 | |
| Earlier maximum setting | Intel UHD | 2.0× | 66–76 | 67–71 | not run | |

The Graphics quality setting offers Auto (default), Maximum and Smooth. Auto starts sharp on the RTX, starts lower on integrated graphics, and lowers or raises the resolution within a second

or two as the frame rate changes.

**What this does not show.** The numbers come from a hidden window with no display synchronisation. They show relative cost, not what the real window feels like on battery or with other programs open.

# Visual quality

Textures come from NASA's public imagery. Rendering adds supersampling on capable graphics cards, 4x multisampling in the glow pass (2x on integrated graphics), full anisotropic filtering, and higher-detail spheres. The camera button saves the current view as a PNG up to about 3,840 pixels wide.

| TEXTURE | BEFORE | NOW | SOURCE |
|---|---|---|---|
| Moon | 1,024 × 512 | 8,192 × 4,096 | NASA Lunar Reconnaissance Orbiter, Scientific Visualization Studio |
| Earth by day | 5,400 × 2,700 | 8,192 × 4,096 | NASA Blue Marble with topography and bathymetry |
| Earth by night | 3,600 × 1,800 | 8,192 × 4,096 | NASA Black Marble city lights |
| Clouds | 1,024 × 512 | 4,096 × 2,048 | NASA Blue Marble cloud map |
| Other planets | 2,048 × 1,024 | 2,048 × 1,024 | Solar System Scope (CC BY 4.0) |

Realism details: cloud shadows fall on the surface and dim the city lights at night, the atmosphere has a thin outer haze, the Moon, Mercury and Mars have relief shading, and the spacecraft model has gridded solar panels, a dish with its feed, a thruster and a magnetometer boom.

# Presenting it professionally

## The thirty-second pitch

"Lambert's problem asks which orbit connects two positions in a given time. I built an application around it. Give it two sightings of a satellite and it finds the orbit. Give it two planets and a date and it plans the trip, with real NASA positions and gravity assists. Every result is checked against textbook and mission figures."

## A six-minute demonstration

**1** **Start page** `0:00`

Let the Earth turn. Point at the readout: real data sources, live clock. Say what the app does in one sentence, then press Start.

**2** **Orbit determination** `0:40`

The assignment data is already solved. Show the two sightings, the 3D orbit, and the elements: a = 28,184.19 km, e = 0.768, i = 20.3°. Say these match the assignment answers.

**3** **Uncertainty** `1:40`

Open "How certain is this orbit?" and run the simulations. Explain that small measurement errors grow along the track.

**4** **Mission design: porkchop plot** `2:30`

Earth to Mars. Explain the dark eye as the cheapest window and note that every pixel is a separate Lambert solution. Mention that launch windows return every 26 months.

**5** **Gravity-assist route and playback** `3:30`

Open the Voyager route and press the play button for the mission playback. Let the flyby close-up play. Say that flybys change speed without fuel.

**6** **Full-gravity check** `4:40`

Press "Check in full solar-system gravity". Explain that the simple model is corrected by small burns and that Voyager needs about 4 m/s in total.

**7** **Close** `5:20`

Show Satellites or Learn for a few seconds, then return to the start page. Offer the single-file hand-in and the validation suite as proof.

## Questions to expect

**Why is Lambert's problem useful?**

It is the core of orbit determination from two positions and of every transfer calculation. The same solver works around the Earth and around the Sun.

**How accurate is it?**

The assignment orbit matches the reference to the digits shown. The Mars case is within 0.01 km/s of a design value, and a full-gravity re-flight lands within 10 km after small corrections.

**What is "patched conics"?**

Each leg is a Sun-centred ellipse and each flyby a planet-centred hyperbola, joined at the planet's sphere of influence. It is the standard first design step. The full-gravity check measures what it leaves out.

**Is the data real?**

Planet positions are NASA/JPL DE440s, satellites are CelesTrak, asteroids are JPL. Offline, the app uses saved copies and says so.

**Could it fly a real mission?**

No. It is a design and teaching tool. Real missions add launch constraints, ephemeris refinement, and operations the app does not model.

**What did you build, and what did you use?**

The solver, validation and application are the project's own. The libraries and data sources are listed in the credits, and AI assistance in building is disclosed there.

**What does it need to run?**

The Windows app needs nothing installed. From source: Python 3.10 or newer and the packages in `requirements.txt`.

**Why did you remove three views?**

They went beyond the topic. Fewer views made the application easier to explain and to run.

## Before you present

- [ ] Plug the laptop in. On battery Windows may switch to the integrated chip.
- [ ] Close other heavy programs and browser windows.
- [ ] Open the app from the desktop shortcut and wait for the start page. The readout should show READY and ONLINE.
- [ ] Press Start once and confirm an orbit appears, then return home with the house icon.
- [ ] In Settings, check Graphics quality is on Auto. Turn on "Show frame rate" if you want to watch it.
- [ ] Keep this report as a PDF and the single-file hand-in on a USB stick as a fallback.
- [ ] If the network is down, say so: the app uses saved data and labels it "saved copy".

## Limits and open items

- **Design tool, not flight software.** Patched conics and the n-body check are first-design methods.
- **Windows and Edge.** The standalone build is Windows only and needs Microsoft Edge installed (it ships with Windows).
- **Graphics memory.** The 8K textures use several hundred megabytes of graphics memory; Smooth quality lowers the load on weaker machines.
- **Not yet profiled by hand.** Performance was measured in a hidden window. A full session on battery power and on the integrated chip alone has not been timed.
- **Two open items** remain in the findings table: the one-off route error and the quick-relaunch behaviour.
- **Reports.** The earlier case-study report and LinkedIn carousel in the `report` folder still describe views that were later removed and show the old interface. Regenerate them before sharing.

## Credits and licences

| ITEM | SOURCE | LICENCE OR TERMS |
|---|---|---|
| Earth, Moon and cloud imagery | NASA Visible Earth, Blue Marble and Black Marble; NASA Scientific Visualization Studio (LRO) | NASA public imagery, credit given |
| Planet textures | Solar System Scope | CC BY 4.0 |
| Planet and Moon positions | NASA/JPL DE440s through Skyfield | Public |
| Asteroids, close approaches, paths | JPL Small-Body Database, CNEOS, Horizons | Public API |
| Satellite element sets | CelesTrak | Public data |
| 3D, charts | Three.js, Plotly | MIT |
| Numerics and packaging | NumPy, SciPy, Skyfield, sgp4, PyInstaller | Open source |
| Method | H. D. Curtis, *Orbital Mechanics for Engineering Students*, Lambert's problem (Algorithm 5.2) | Textbook reference |

This application was built with the assistance of Claude Code from Anthropic. The author directed the work, chose the scope and design, and tested the results.

# Appendix

## Project layout

| FOLDER | CONTENTS |
|---|---|
| `src/` | All Python modules: physics, server, window launcher, course GUI |
| `web/` | The interface: HTML, CSS, JavaScript, NASA textures |
| `course/` | The assignment hand-in, `lambert_problem_solver.py`, generated from the modules |
| `tools/` | Build scripts for the Windows app, the single file, and the icon |
| `tests/` | The pytest suite |
| `report/` | Case-study report, carousel, LinkedIn post, and this report |

## Commands

| TASK | COMMAND |
|---|---|
| Start the app | `run_studio.bat` or the desktop shortcut |
| Run all tests | `python -m pytest` |
| Run the validation suite | `python src/validation.py` |
| Rebuild the single-file hand-in | `python tools/build_single_file.py` |
| Build the Windows app | `python tools/build_exe.py` |
| Regenerate the icon | `python tools/make_icon.py` |

Lambert Studio, version 1.0. Figures in this report come from the project's own validation run, test run, git history and measurements made on 3 and 4 October 2026.